

Problem(1 – 2): Round Robin

Source code:

```
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;
#ifdef ONLINE_JUDGE
#define debug(x) cerr << #x << " = " << x << endl
#else
#define debug(x)
#endif

using ll = long long;

void Solve()
{
    int n, tq;
    cin >> n;

    vector<int> at(n), bt(n), rt(n);
    vector<int> ct(n), tat(n), wt(n);

    for (int i = 0; i < n; i++) {
        cin >> at[i] >> bt[i];
        rt[i] = bt[i];
    }

    cin >> tq;

    queue<int> q;
    vector<bool> vis(n, false);

    int time = 0, completed = 0;

    int idx = 0;
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);

    sort(order.begin(), order.end(), [&](int a, int b) {
        return at[a] < at[b];
    });

    int ptr = 0;
```

```

while (completed < n) {

    while (ptr < n && at[order[ptr]] <= time) {
        q.push(order[ptr]);
        vis[order[ptr]] = true;
        ptr++;
    }

    if (q.empty()) {
        time = at[order[ptr]];
        continue;
    }

    int p = q.front();
    q.pop();

    int exec = min(tq, rt[p]);

    rt[p] -= exec;
    time += exec;

    while (ptr < n && at[order[ptr]] <= time) {
        q.push(order[ptr]);
        vis[order[ptr]] = true;
        ptr++;
    }

    if (rt[p] > 0) {
        q.push(p);
    }
    else {
        completed++;
        ct[p] = time;
    }
}

double avgWaitingTime = 0, avgTurnAroundTime = 0;

for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avgWaitingTime += wt[i];
}

```

```

        avgTurnAroundTime += tat[i];
    }

    cout << "PID AT BT CT TAT WT\n";

    for (int i = 0; i < n; i++) {
        cout << "P" << i + 1 << " "
            << at[i] << " "
            << bt[i] << " "
            << ct[i] << " "
            << tat[i] << " "
            << wt[i] << endl;
    }

    cout << "\nAverage Waiting Time = " << avgWaitingTime / n << endl;
    cout << "Average Turnaround Time = " << avgTurnAroundTime / n << endl;
}

int32_t main()
{
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    ll t = 1;
    // cin >> t;
    while (t--)
        Solve();
    return 0;
}

```

Sample Input:

Input:
Copy

```

4
0 5
1 3
2 4
3 7
3

```

Sample Output:

```
Received Output: Set Copy
PID AT BT CT TAT WT
P1 0 5 14 14 9
P2 1 3 6 5 2
P3 2 4 15 13 9
P4 3 7 19 16 9

Average Waiting Time = 7.25
Average Turnaround Time = 12
```

Problem (2 – 1): FIFO

Source code:

```
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;
#ifdef ONLINE_JUDGE
#define debug(x) cerr << #x << " = " << x << endl
#else
#define debug(x)
#endif

using ll = long long;

void Solve()
{
    int n, f;

    cin >> n;

    vector<int> pages(n);
    for (int i = 0; i < n; i++)
        cin >> pages[i];

    cin >> f;
```

```

queue<int> q;
unordered_set<int> s;

int hit = 0, fault = 0;

for (int i = 0; i < n; i++) {

    if (s.find(pages[i]) != s.end()) {

        hit++;
        cout << "Page " << pages[i] << " : Hit\n";
    }
    else {

        fault++;

        if (q.size() == f) {
            int x = q.front();
            q.pop();
            s.erase(x);
        }

        q.push(pages[i]);
        s.insert(pages[i]);

        cout << "Page " << pages[i] << " : Fault\n";
    }
}

double hitRatio = (double)hit / n;
double missRatio = (double)fault / n;

cout << "\nTotal Pages = " << n << endl;
cout << "Page Hits = " << hit << endl;
cout << "Page Faults = " << fault << endl;

cout << "Hit Ratio = " << hitRatio << endl;
cout << "Miss Ratio = " << missRatio << endl;
}

int32_t main()
{
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    ll t = 1;

```

```
// cin >> t;  
while (t--)  
    Solve();  
return 0;  
}
```

Sample Input:

```
Input:                                     Copy  
7  
1 3 0 3 5 6 3  
3
```

Sample Output:

```
Received Output:                           Set Copy  
Page 1 : Fault  
Page 3 : Fault  
Page 0 : Fault  
Page 3 : Hit  
Page 5 : Fault  
Page 6 : Fault  
Page 3 : Fault  
  
Total Pages = 7  
Page Hits = 1  
Page Faults = 6  
Hit Ratio = 0.142857  
Miss Ratio = 0.857143
```